

Program 09:- Write a program to implement fractional knapsack problem using Greedy strategy.

```
#include <stdio.h>
#include <stdlib.h>
struct Item {
    int profit, weight;
};
int cmp(const void* a, const void* b) {
    struct Item* itemA = (struct Item*)a;
    struct Item* itemB = (struct Item*)b;
    double r1 = (double)itemA->profit / itemA->weight;
    double r2 = (double)itemB->profit / itemB->weight;
    return r1 < r2 ? 1 : -1;
}
double fractionalKnapsack(int W, struct Item arr[], int N) {
    qsort(arr, N, sizeof(struct Item), cmp);
    double finalValue = 0.0;
    for (int i = 0; i < N; i++) {
        if (arr[i].weight <= W) {
            W -= arr[i].weight;
            finalValue += arr[i].profit;
        } else {
            finalValue += arr[i].profit * ((double)W / arr[i].weight);
            break;
        }
    }
    return finalValue;
}
int main() {
    int N, W;
    printf("Enter the number of items: ");
    scanf("%d", &N);
    struct Item arr[N];
    printf("Enter the capacity of knapsack: ");
    scanf("%d", &W);
    for (int i = 0; i < N; i++) {
        printf("Enter profit and weight for item %d: ", i + 1);
        scanf("%d %d", &arr[i].profit, &arr[i].weight);
    }
    printf("Maximum profit in Knapsack = %.2f\n", fractionalKnapsack(W, arr, N));
    return 0;
}
```

```
Enter the number of items: 3
Enter the capacity of knapsack: 50
Enter profit and weight for item 1: 60 10
Enter profit and weight for item 2: 100 20
Enter profit and weight for item 3: 120 30
Maximum profit in Knapsack = 240.00
```

=== Code Execution Successful ===

Program 10

Program 10:- Write a program to find the minimum cost spanning tree using Kruskal's algorithm.

```
#include <stdio.h>
#include <stdlib.h>
int comparator(const void* p1, const void* p2) {
    const int(*x)[3] = p1;
    const int(*y)[3] = p2;
    return (*x)[2] - (*y)[2];
}
void makeSet(int parent[], int rank[], int n) {
    for (int i = 0; i < n; i++) {
        parent[i] = i;
        rank[i] = 0;
    }
}
int findParent(int parent[], int component) {
    if (parent[component] == component)
        return component;
    return parent[component] = findParent(parent, parent[component]);
}
void unionSet(int u, int v, int parent[], int rank[], int n) {
    u = findParent(parent, u);
    v = findParent(parent, v);
    if (rank[u] < rank[v]) {
        parent[u] = v;
    } else if (rank[u] > rank[v]) {
        parent[v] = u;
    } else {
        parent[v] = u;
        rank[u]++;
    }
}
void kruskalAlgo(int n, int edge[n][3]) {
    qsort(edge, n, sizeof(edge[0]), comparator);
    int parent[n], rank[n];
    makeSet(parent, rank, n);
    int minCost = 0;
    printf("Following are the edges in the constructed MST\n");
    for (int i = 0; i < n; i++) {
        int v1 = findParent(parent, edge[i][0]);
        int v2 = findParent(parent, edge[i][1]);
        int wt = edge[i][2];
        if (v1 != v2) {
            unionSet(v1, v2, parent, rank, n);
            minCost += wt;
            printf("%d -- %d == %d\n", edge[i][0], edge[i][1], wt);
        }
    }
    printf("Minimum Cost Spanning Tree: %d\n", minCost);
}
```

```

}
int main() {
    int n;
    printf("Enter the number of edges: ");
    scanf("%d", &n);
    int edge[n][3];
    printf("Enter each edge in the format (u v weight):\n");
    for (int i = 0; i < n; i++) {
        printf("Edge %d: ", i + 1);
        scanf("%d %d %d", &edge[i][0], &edge[i][1], &edge[i][2]);
    }
    kruskalAlgo(n, edge);
    return 0;
}

```

```

Enter the number of edges: 5
Enter each edge in the format (u v weight):
Edge 1: 0 1 10
Edge 2: 0 2 6
Edge 3: 0 3 5
Edge 4: 1 3 15
Edge 5: 2 3 4
Following are the edges in the constructed MST
2 -- 3 == 4
0 -- 3 == 5
0 -- 1 == 10
Minimum Cost Spanning Tree: 19

=== Code Execution Successful ===

```